

```
campo.setCustomValidity("Mensagem...");
```

Validação de Formulários em JS

Parte 2 — mensagens de erro personalizadas com a API de validação do HTML5.

Por onde vamos passar

01 Validação de formulários com mensagens personalizadas

02 Exercício de fixação

03 Referências bibliográficas

01

— Mensagens personalizadas

Reaproveitando o HTML5 e controlando o texto do erro.

■ PROBLEMA 2

Ampliando o formulário da aula anterior

Ao formulário do problema anterior, adicione campos para o usuário informar sua **idade** e seu **e-mail** — exibindo [mensagens de erro personalizadas](#), mas considerando as validações do HTML5.

>
0 Idade deve ser um
valor positivo.

@ E-mail deve ser um **valor válido.**

req Ambos são **obrigatórios.**

O que cada camada consegue validar

HTML5

Restrições de cada campo

Obrigatoriedade, faixa de valores, tipo, tamanho —
`required`, `min`, `max`, `maxlength`...

JAVASCRIPT

Regras entre campos

O que o HTML não expressa. Ex.: garantir que o campo
`nome` seja diferente do campo `sobrenome`.

■ 0 DESAFIO

Não refazer o que o HTML já valida — reaproveitar

As restrições embutidas (`required`, `min`, `max...`) não devem ser duplicadas em JavaScript.
Como resolver?

Dois passos básicos

1 Desabilitar a validação automática

Desligar a checagem automática das restrições definidas na página HTML.

2 Verificar as restrições via script

Escrever um script que confira, ele mesmo, se as restrições do HTML foram obedecidas — usando a API de validação.

A propriedade `noValidate`

Atributo booleano novo do HTML5. Desliga a validação automática do formulário.

Boa prática: desligar via JavaScript (DOM). Mesmo assim, ainda é possível verificar as restrições do HTML por script.

```
window.onload = iniciarPagina;  
function iniciarPagina() {  
    document.getElementById("form").noValidate  
    = true;  
    // ...  
}
```


■ PASSO 2 • VERIFICAR

A API de validação do HTML5

A **Constraint Validation** oferece propriedades e métodos, disponíveis em todos os elementos `input`, para checar as restrições do HTML por conta própria.

■ O OBJETO DE ESTADO

A propriedade **validity**

Presente em todo `input`, verifica se o valor satisfaz **todas as restrições** do HTML. É um **objeto**, com uma propriedade para cada tipo de restrição.

Este campo é válido?

```
function validarEmail() {  
  var campoEmail;  
  campoEmail = document.getElementById("email");  
  if (!campoEmail.validity.valid) {  
    alert("E-mail inválido!");  
    return false;  
  }  
  return true;  
}
```

`valid` pertence ao objeto `validity` e verifica se **todas** as restrições HTML do campo foram obedecidas.

Se alguma restrição do e-mail falhar, a função retorna `false` e o envio é barrado.

■ O OBJETO VALIDITY POR DENTRO

Uma propriedade para cada restrição

<code>valueMissing</code>	Sem valor, tendo <code>required</code> .
<code>patternMismatch</code>	Não atende ao <code>pattern</code> .
<code>rangeUnderflow</code>	Menor que o <code>min</code> .
<code>stepMismatch</code>	Não atende à escala do <code>step</code> .
<code>valid</code>	Todas as restrições satisfeitas.

<code>typeMismatch</code>	Não atende ao <code>type</code> especificado.
<code>tooLong</code>	Excede o <code>maxLength</code> .
<code>rangeOverflow</code>	Maior que o <code>max</code> .
<code>customError</code>	Há mensagem personalizada definida.

■ UMA API EXTENSÍVEL

Você define suas próprias mensagens de erro

Ao definir uma mensagem de erro para um `input`, a propriedade `valid` daquele elemento passa a ser `false` **automaticamente**.

Redefinir e acessar a mensagem

`setCustomValidity(msg)`

Redefine a mensagem de erro padrão do input. O argumento é a **nova mensagem** (string).

`validationMessage`

Propriedade que **acessa a mensagem** de erro atual de um elemento input.

```
campoEmail.setCustomValidity("Informe um e-mail válido!");  
campoEmail.setCustomValidity(""); // limpa a mensagem
```

■ EXIBINDO O ERRO

O método `reportValidity()` do formulário

Verifica se o formulário é válido e exibe a mensagem de erro no navegador — no mesmo formato da mensagem padrão. Assim, o texto trocado com `setCustomValidity()` aparece nativamente.

```
function validarDados(f) {  
  let elementos = f.elements;  
  // limpar mensagens customizadas  
  for (let i = 0; i < elementos.length; i++) {  
    elementos[i].setCustomValidity("");  
  }  
  validarCamposComuns(f);  
  return f.reportValidity();  
}
```

02

— Exercício de fixação

Personalize as mensagens de erro do HTML5.

Cadastro com mensagens personalizadas

Formulário de **e-mail**, **senha** e **confirmação** — trocando as mensagens de erro padrão do HTML5 por mensagens suas.

HTML5

Todos os campos são **obrigatórios**.

HTML5

O **e-mail** deve ser válido.

JS

Senha e confirmação devem ser **iguais**.

JS

Só envia se **todos** os dados forem válidos.

Para aprofundar

MAZZA, L. **HTML5 e CSS3 — Domine a web do futuro**. São Paulo: Casa do Código, 2012. 197 p.

SILVA, M. S. **HTML5 — A linguagem de marcação que revolucionou a Web**. São Paulo: Novatec, 2011. 320 p.

SILVA, M. S. **JavaScript: Guia do Programador**. São Paulo: Novatec, 2010. 604 p.

W3SCHOOLS. Disponível em: <http://www.w3schools.com>